

18.1-5

Describe the data structure that results if each black node in a red-black tree absorbs its red children, incorporating their children with its own.

18.2 Basic operations on B-trees

This section presents the details of the operations B-TREE-SEARCH, B-TREE-CREATE, and B-TREE-INSERT. These procedures observe two conventions:

- The root of the B-tree is always in main memory, so that no procedure ever needs to perform a DISK-READ on the root. If any changes to the root node occur, however, then DISK-WRITE must be called on the root.
- Any nodes that are passed as parameters must already have had a DISK-READ operation performed on them.

The procedures are all “one-pass” algorithms that proceed downward from the root of the tree, without having to back up.

Searching a B-tree

Searching a B-tree is much like searching a binary search tree, except that instead of making a binary, or “two-way,” branching decision at each node, the search makes a multiway branching decision according to the number of the node’s children. More precisely, at each internal node x , the search makes an $(x.n + 1)$ -way branching decision.

The procedure B-TREE-SEARCH generalizes the TREE-SEARCH procedure defined for binary search trees on page 316. It takes as input a pointer to the root node x of a subtree and a key k to be searched for in that subtree. The top-level call is thus of the form B-TREE-SEARCH($T.root$, k). If k is in the B-tree, then B-TREE-SEARCH returns the ordered pair (y, i) consisting of a node y and an index i such that $y.key_i = k$. Otherwise, the procedure returns NIL.

```

B-TREE-SEARCH( $x, k$ )
1  $i = 1$ 
2 while  $i \leq x.n$  and  $k > x.key_i$ 
3    $i = i + 1$ 
4 if  $i \leq x.n$  and  $k == x.key_i$ 
5   return ( $x, i$ )
6 elseif  $x.leaf$ 
7   return NIL
8 else DISK-READ( $x.c_i$ )
9   return B-TREE-SEARCH( $x.c_i, k$ )

```

Using a linear-search procedure, lines 1–3 of B-TREE-SEARCH find the smallest index i such that $k \leq x.key_i$, or else they set i to $x.n + 1$. Lines 4–5 check to see whether the search has discovered the key, returning if it has. Otherwise, if x is a leaf, then line 7 terminates the search unsuccessfully, and if x is an internal node, lines 8–9 recurse to search the appropriate subtree of x , after performing the necessary DISK-READ on that child. Figure 18.1 illustrates the operation of B-TREE-SEARCH. The blue nodes are those examined during a search for the key R .

As in the TREE-SEARCH procedure for binary search trees, the nodes encountered during the recursion form a simple path downward from the root of the tree. The B-TREE-SEARCH procedure therefore accesses $O(h) = O(\log_t n)$ disk blocks, where h is the height of the B-tree and n is the number of keys in the B-tree. Since $x.n < 2t$, the **while** loop of lines 2–3 takes $O(t)$ time within each node, and the total CPU time is $O(th) = O(t \log_t n)$.

Creating an empty B-tree

To build a B-tree T , first use the B-TREE-CREATE procedure on the next page to create an empty root node and then call the B-TREE-INSERT procedure on page 508 to add new keys. Both of these

procedures use an auxiliary procedure `ALLOCATE-NODE`, whose pseudocode we omit and which allocates one disk block to be used as a new node in $O(1)$ time. A node created by `ALLOCATE-NODE` requires no `DISK-READ`, since there is as yet no useful information stored on the disk for that node. `B-TREE-CREATE` requires $O(1)$ disk operations and $O(1)$ CPU time.

B-TREE-CREATE(T)

```
1  $x = \text{ALLOCATE-NODE}()$ 
2  $x.\text{leaf} = \text{TRUE}$ 
3  $x.n = 0$ 
4  $\text{DISK-WRITE}(x)$ 
5  $T.\text{root} = x$ 
```

Inserting a key into a B-tree

Inserting a key into a B-tree is significantly more complicated than inserting a key into a binary search tree. As with binary search trees, you search for the leaf position at which to insert the new key. With a B-tree, however, you cannot simply create a new leaf node and insert it, as the resulting tree would fail to be a valid B-tree. Instead, you insert the new key into an existing leaf node. Since you cannot insert a key into a leaf node that is full, you need an operation that *splits* a full node y (having $2t - 1$ keys) around its *median key* $y.\text{key}_t$ into two nodes having only $t - 1$ keys each. The median key moves up into y 's parent to identify the dividing point between the two new trees. But if y 's parent is also full, you must split it before you can insert the new key, and thus you could end up splitting full nodes all the way up the tree.

To avoid having to go back up the tree, just split every full node you encounter as you go down the tree. In this way, whenever you need to split a full node, you are assured that its parent is not full. Inserting a key into a B-tree then requires only a single pass down the tree from the root to a leaf.

Splitting a node in a B-tree

The procedure B-TREE-SPLIT-CHILD on the facing page takes as input a *nonfull* internal node x (assumed to reside in main memory) and an index i such that $x.c_i$ (also assumed to reside in main memory) is a *full* child of x . The procedure splits this child in two and adjusts x so that it has an additional child. To split a full root, you first need to make the root a child of a new empty root node, so that you can use B-TREE-SPLIT-CHILD. The tree thus grows in height by 1: splitting is the only means by which the tree grows taller.

```

B-TREE-SPLIT-CHILD( $x, i$ )
1  $y = x.c_i$                                 // full node to split
2  $z = \text{ALLOCATE-NODE}()$                     //  $z$  will take half of  $y$ 
3  $z.\text{leaf} = y.\text{leaf}$ 
4  $z.n = t - 1$ 
5 for  $j = 1$  to  $t - 1$                         //  $z$  gets  $y$ 's greatest keys ...
6    $z.\text{key}_j = y.\text{key}_{j+t}$ 
7 if not  $y.\text{leaf}$ 
8   for  $j = 1$  to  $t$                             // ... and its corresponding children
9      $z.c_j = y.c_{j+t}$ 
10  $y.n = t - 1$                                 //  $y$  keeps  $t - 1$  keys
11 for  $j = x.n + 1$  downto  $i + 1$               // shift  $x$ 's children to the right ...
12    $x.c_{j+1} = x.c_j$ 
13  $x.c_{i+1} = z$                                 // ... to make room for  $z$  as a child
14 for  $j = x.n$  downto  $i$                         // shift the corresponding keys in  $x$ 
15    $x.\text{key}_{j+1} = x.\text{key}_j$ 
16  $x.\text{key}_i = y.\text{key}_t$                             // insert  $y$ 's median key
17  $x.n = x.n + 1$                                 //  $x$  has gained a child
18 DISK-WRITE( $y$ )
19 DISK-WRITE( $z$ )
20 DISK-WRITE( $x$ )

```

Figure 18.5 illustrates how a node splits. B-TREE-SPLIT-CHILD splits the full node $y = x.c_i$ about its median key (S in the figure), which

moves up into y 's parent node x . Those keys in y that are greater than the median key move into a new node z , which becomes a new child of x .

B-TREE-SPLIT-CHILD works by straightforward cutting and pasting. Node x is the parent of the node y being split, which is x 's i th child (set in line 1). Node y originally has $2t$ children and $2t - 1$ keys, but splitting reduces y to t children and $t - 1$ keys. The t largest children and $t - 1$ keys of node y move over to node z , which becomes a new child of x , positioned just after y in x 's table of children. The median key of y moves up to become the key in node x that separates the pointers to nodes y and z .

Lines 2–9 create node z and give it the largest $t - 1$ keys and, if y and z are internal nodes, the corresponding t children of y . Line 10 adjusts the key count for y . Then, lines 11–17 shift keys and child pointers in x to the right in order to make room for x 's new child, insert z as a new child of x , move the median key from y up to x in order to separate y from z , and adjust x 's key count. Lines 18–20 write out all modified disk blocks. The CPU time used by **B-TREE-SPLIT-CHILD** is $\Theta(t)$, due to the **for** loops in lines 5–6 and 8–9. (The **for** loops in lines 11–12 and 14–15 also run for $O(t)$ iterations.) The procedure performs $O(1)$ disk operations.

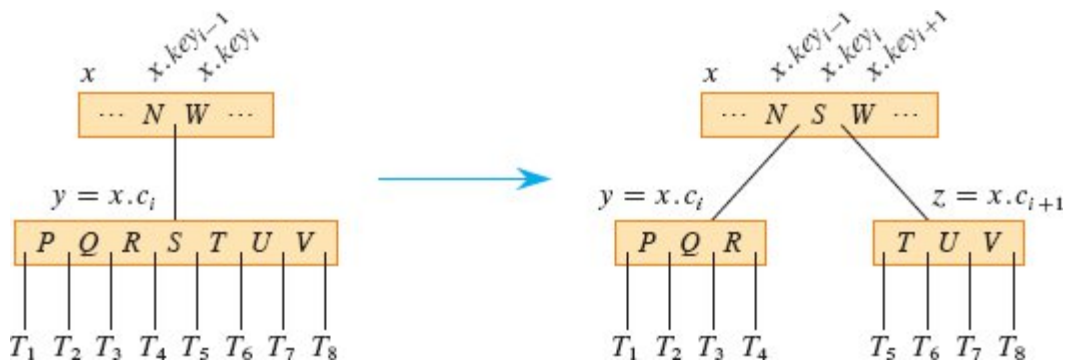


Figure 18.5 Splitting a node with $t = 4$. Node $y = x.c_i$ splits into two nodes, y and z , and the median key S of y moves up into y 's parent.

Inserting a key into a B-tree in a single pass down the tree

Inserting a key k into a B-tree T of height h requires just a single pass down the tree and $O(h)$ disk accesses. The CPU time required is $O(th) = O(t \log_t n)$. The B-TREE-INSERT procedure uses B-TREE-SPLIT-CHILD to guarantee that the recursion never descends to a full node. If the root is full, B-TREE-INSERT splits it by calling the procedure B-TREE-SPLIT-ROOT on the facing page.

B-TREE-INSERT(T, k)

```
1  $r = T.root$ 
2 if  $r.n == 2t - 1$ 
3    $s = \text{B-TREE-SPLIT-ROOT}(T)$ 
4   B-TREE-INSERT-NONFULL( $s, k$ )
5 else B-TREE-INSERT-NONFULL( $r, k$ )
```

B-TREE-INSERT works as follows. If the root is full, then line 3 calls B-TREE-SPLIT-ROOT in line 3 to split it. A new node s (with two children) becomes the root and is returned by B-TREE-SPLIT-ROOT. Splitting the root, illustrated in Figure 18.6, is the only way to increase the height of a B-tree. Unlike a binary search tree, a B-tree increases in height at the top instead of at the bottom. Regardless of whether the root split, B-TREE-INSERT finishes by calling B-TREE-INSERT-NONFULL to insert key k into the tree rooted at the nonfull root node, which is either the new root (the call in line 4) or the original root (the call in line 5).

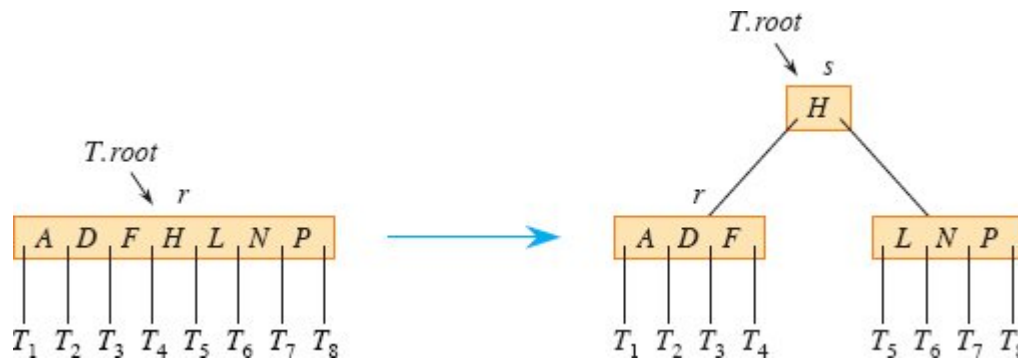


Figure 18.6 Splitting the root with $t = 4$. Root node r splits in two, and a new root node s is created. The new root contains the median key of r and has the two halves of r as children. The B-tree grows in height by one when the root is split. A B-tree's height increases only when the root splits.

B-TREE-SPLIT-ROOT(T)

```

1  $s = \text{ALLOCATE-NODE}()$ 
2  $s.\text{leaf} = \text{FALSE}$ 
3  $s.n = 0$ 
4  $s.c_1 = T.\text{root}$ 
5  $T.\text{root} = s$ 
6 B-TREE-SPLIT-CHILD( $s, 1$ )
7 return  $s$ 

```

The auxiliary procedure B-TREE-INSERT-NONFULL on page 511 inserts key k into node x , which is assumed to be nonfull when the procedure is called. B-TREE-INSERT-NONFULL recurses as necessary down the tree, at all times guaranteeing that the node to which it recurses is not full by calling B-TREE-SPLIT-CHILD as necessary. The operation of B-TREE-INSERT and the recursive operation of B-TREE-INSERT-NONFULL guarantee that this assumption is true.

Figure 18.7 illustrates the various cases of how B-TREE-INSERT-NONFULL inserts a key into a B-tree. Lines 3–8 handle the case in which x is a leaf node by inserting key k into x , shifting to the right all keys in x that are greater than k . If x is not a leaf node, then k should

go into the appropriate leaf node in the subtree rooted at internal node x . Lines 9–11 determine the child $x.c_i$ to which the recursion descends. Line 13 detects whether the recursion would descend to a full child, in which case line 14 calls **B-TREE-SPLIT-CHILD** to split that child into two nonfull children, and lines 15–16 determine which of the two children is the correct one to descend to. (Note that **DISK-READ**($x.c_i$) is not needed after line 16 increments i , since the recursion descends in this case to a child that was just created by **B-TREE-SPLIT-CHILD**.) The net effect of lines 13–16 is thus to guarantee that the procedure never recurses to a full node. Line 17 then recurses to insert k into the appropriate subtree.

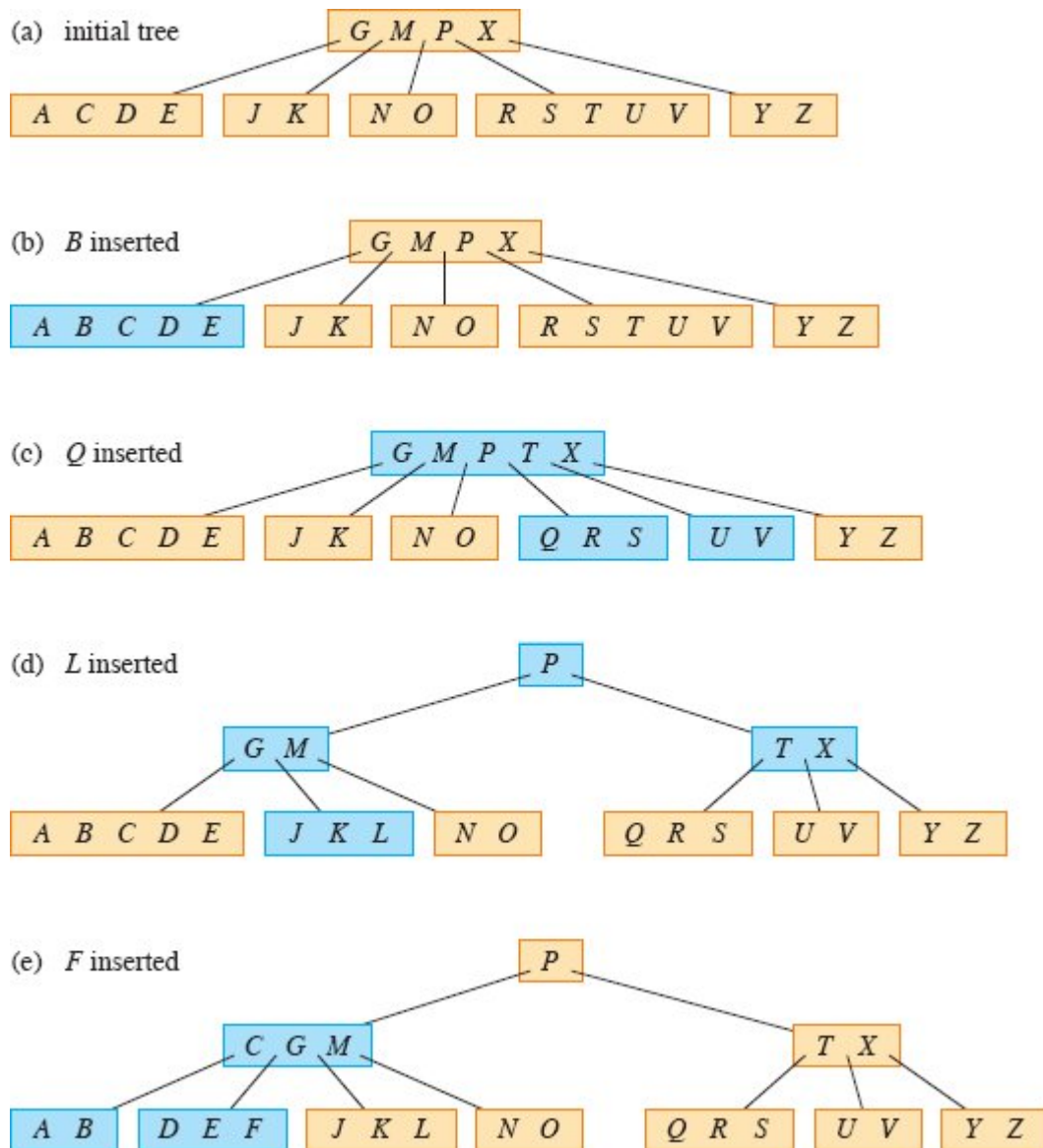


Figure 18.7 Inserting keys into a B-tree. The minimum degree t for this B-tree is 3, so that a node can hold at most 5 keys. Blue nodes are modified by the insertion process. **(a)** The initial tree for this example. **(b)** The result of inserting *B* into the initial tree. This case is a simple insertion into a leaf node. **(c)** The result of inserting *Q* into the previous tree. The node *RSTUV* splits into two nodes containing *RS* and *UV*, the key *T* moves up to the root, and *Q* is inserted in the leftmost of the two halves (the *RS* node). **(d)** The result of inserting *L* into the previous tree. The root splits right away, since it is full, and the B-tree grows in height by one. Then *L* is inserted into the leaf containing *JK*. **(e)** The result of inserting *F* into the previous tree. The node *ABCDE* splits before *F* is inserted into the rightmost of the two halves (the *DE* node).

B-TREE-INSERT-NONFULL(x, k)

```
1  $i = x.n$ 
2 if  $x.leaf$                                 // inserting into a leaf?
3   while  $i \geq 1$  and  $k < x.key_i$           // shift keys in  $x$  to make room for  $k$ 
4      $x.key_{i+1} = x.key_i$ 
5      $i = i - 1$ 
6    $x.key_{i+1} = k$                             // insert key  $k$  in  $x$ 
7    $x.n = x.n + 1$                             // now  $x$  has 1 more key
8   DISK-WRITE( $x$ )
9 else while  $i \geq 1$  and  $k < x.key_i$         // find the child where  $k$  belongs
10    $i = i - 1$ 
11    $i = i + 1$ 
12   DISK-READ( $x.c_i$ )
13   if  $x.c_i.n == 2t - 1$                     // split the child if it's full
14     B-TREE-SPLIT-CHILD( $x, i$ )
15     if  $k > x.key_i$                         // does  $k$  go into  $x.c_i$  or  $x.c_{i+1}$ ?
16        $i = i + 1$ 
17   B-TREE-INSERT-NONFULL( $x.c_i, k$ )
```

For a B-tree of height h , B-TREE-INSERT performs $O(h)$ disk accesses, since only $O(1)$ DISK-READ and DISK-WRITE operations occur at each level of the tree. The total CPU time used is $O(t)$ in each level of the tree, or $O(th) = O(t \log_t n)$ overall. Since B-TREE-INSERT-NONFULL is tail-recursive, you can instead implement it with a **while** loop, thereby demonstrating that the number of blocks that need to be in main memory at any time is $O(1)$.

Exercises

18.2-1

Show the results of inserting the keys

$F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B, X, Y, D, Z, E$

in order into an empty B-tree with minimum degree 2. Draw only the configurations of the tree just before some node must split, and also draw the final configuration.

18.2-2

Explain under what circumstances, if any, redundant DISK-READ or DISK-WRITE operations occur during the course of executing a call to B-TREE-INSERT. (A redundant DISK-READ is a DISK-READ for a block that is already in memory. A redundant DISK-WRITE writes to disk a block of information that is identical to what is already stored there.)

18.2-3

Professor Bunyan asserts that the B-TREE-INSERT procedure always results in a B-tree with the minimum possible height. Show that the professor is mistaken by proving that with $t = 2$ and the set of keys $\{1, 2, \dots, 15\}$, there is no insertion sequence that results in a B-tree with the minimum possible height.

★ 18.2-4

If you insert the keys $\{1, 2, \dots, n\}$ into an empty B-tree with minimum degree 2, how many nodes does the final B-tree have?

18.2-5

Since leaf nodes require no pointers to children, they could conceivably use a different (larger) t value than internal nodes for the same disk block size. Show how to modify the procedures for creating and inserting into a B-tree to handle this variation.

18.2-6

Suppose that you implement B-TREE-SEARCH to use binary search rather than linear search within each node. Show that this change makes the required CPU time $O(\lg n)$, independent of how t might be chosen as a function of n .

18.2-7

Suppose that disk hardware allows you to choose the size of a disk block arbitrarily, but that the time it takes to read the disk block is $a+bt$, where a and b are specified constants and t is the minimum degree for a B-tree using blocks of the selected size. Describe how to choose t so as to minimize (approximately) the B-tree search time. Suggest an optimal value of t for the case in which $a = 5$ milliseconds and $b = 10$ microseconds.

18.3 Deleting a key from a B-tree

Deletion from a B-tree is analogous to insertion but a little more complicated, because you can delete a key from any node—not just a leaf—and when you delete a key from an internal node, you must rearrange the node's children. As in insertion, you must guard against deletion producing a tree whose structure violates the B-tree properties. Just as a node should not get too big due to insertion, a node must not get too small during deletion (except that the root is allowed to have fewer than the minimum number $t - 1$ of keys). And just as a simple insertion algorithm might have to back up if a node on the path to where the key is to be inserted is full, a simple approach to deletion might have to back up if a node (other than the root) along the path to where the key is to be deleted has the minimum number of keys.

The procedure B-TREE-DELETE deletes the key k from the subtree rooted at x . Unlike the procedures TREE-DELETE on page 325 and RB-DELETE on page 348, which are given the node to delete—presumably as the result of a prior search—B-TREE-DELETE combines the search for key k with the deletion process. Why do we combine search and deletion in B-TREE-DELETE? Just as B-TREE-INSERT prevents any node from becoming overfull (having more than $2t - 1$ keys) while making a single pass down the tree, B-TREE-DELETE prevents any node from becoming underfull (having fewer than $t - 1$ keys) while also making a single pass down the tree, searching for and ultimately deleting the key.

To prevent any node from becoming underfull, the design of B-TREE-DELETE guarantees that whenever it calls itself recursively on a